

Lecture 2: Parameters for Evaluating Algorithms

*Instructor: Sourav Chakraborty, Sanjukta Roy**Scribe: Debabrata, Pritam*

1 Finding the Maximum Element

Given n numbers, we want to find the maximum element.

1.1 Algorithm (Using Array)

```
max = a[0]
for (int i = 0; i < n; i++) {
    if (a[i] > max) {
        max = a[i];
    }
}
```

1.2 Time Complexity Analysis

- Worst case time complexity: $O(n)$
- Average case time complexity: $O(n)$
- Best case time complexity: $O(1)$

2 Data Structures for Maintaining Maximum

We compare different data structures based on building cost, finding the maximum, insertion, and deletion.

2.1 Sorted Insertion in a Linked List

Sorted insertion in a linked list means that elements are inserted in such a way that the list always remains sorted (either in ascending or descending order). For every insertion, the correct position is found by traversing the list, and the new element is placed at that position.

- **Building the data structure:** $\mathcal{O}(n^2)$

Each of the n insertions may require traversing up to n nodes in the worst case.

- **Finding the maximum:** $\mathcal{O}(1)$

In a descending sorted list, the maximum element is stored at the head.

- **Adding a new element:** $\mathcal{O}(n)$

Traversal is required to find the correct position for insertion.

- **Removing an element:** $\mathcal{O}(n)$

Traversal is needed to locate the element before deletion.

2.2 Using an Array (Without Sorting)

An unsorted array stores elements without maintaining any order. Elements are added at the end, which makes insertion fast, but operations such as searching, finding the maximum, or removing a specific element require scanning the array.

- **Building:** $\mathcal{O}(n)$

Insert n elements sequentially into the array.

- **Finding maximum:** $\mathcal{O}(n)$

Traverse all elements to find the largest value.

- **Adding an element:** $\mathcal{O}(1)$

Insert the element at the end of the array.

- **Removing an element:** $\mathcal{O}(n)$

Locate the element and shift the remaining elements.

2.3 Using an Array (With Sorting)

A sorted array stores elements in a specific order (ascending or descending). Sorting allows fast access to extreme elements (such as the minimum or maximum), but insertion and deletion require shifting elements to maintain the sorted order.

- **Building the sorted array:** $\mathcal{O}(n \log n)$

Sort n elements using an efficient sorting algorithm such as **Quick Sort** or **Merge Sort**.

- **Finding maximum:** $\mathcal{O}(1)$

The maximum element is located at the end (or beginning) of the sorted array.

- **Adding an element:** $\mathcal{O}(n)$

Find the correct position and shift elements to maintain sorted order.

- **Removing an element:** $\mathcal{O}(n)$

Remove the element and shift the remaining elements.

2.4 Balanced Binary Search Tree

A Balanced Binary Search Tree is a binary search tree in which the height is kept close to minimal by maintaining balance after insertions and deletions. For every node, the heights of its left and right subtrees differ by at most a constant (usually 1).

Examples include AVL Trees and Red–Black Trees.

- **Building:** $\mathcal{O}(n \log n)$

Inserting n elements one by one, each insertion taking $\mathcal{O}(\log n)$.

- **Finding Maximum:** $\mathcal{O}(\log n)$

Traverse the rightmost path of the tree whose height is $\mathcal{O}(\log n)$.

- **Adding an Element:** $\mathcal{O}(\log n)$

Insert the element and rebalance the tree using rotations.

- **Removing an Element:** $\mathcal{O}(\log n)$

Delete the node and rebalance to maintain logarithmic height.

2.5 Using a Max Heap

A Max Heap is a complete binary tree in which the value at every node is greater than or equal to the values of its children. As a result, the maximum element is always stored at the root.

- **Building the heap:** $\mathcal{O}(n)$

Construct the heap from an array using the bottom-up heapify method.

- **Finding maximum:** $\mathcal{O}(1)$

The maximum element is always at the root.

- **Adding an element:** $\mathcal{O}(\log n)$

Insert at the end and restore the heap property by percolating up.

- **Removing the maximum:** $\mathcal{O}(\log n)$

Remove the root and restore the heap property by percolating down.

3 Parameters to judge a sorting algorithm:

Sorting algorithms are evaluated based on the number of comparisons and swaps.

3.1 Time Complexity Measures

- Worst-case time complexity
- Average-case time complexity
- Best-case time complexity

- Expected time complexity (non-deterministic path for input)
- Good-case time complexity (number of swaps required is low)
- Natural time complexity

3.2 Space Complexity

- **In-place sorting:** An algorithm is in-place if it rearranges or modifies the input using only a small, constant amount of extra memory

3.3 Stability of a Sorting Algorithm

Given an input array $a_1, a_2, \dots, a_n$, a sorting algorithm is said to be **stable** if :

- If $a_i > a_j$, then a_j appears before a_i in the sorted array.
- If $a_i = a_j$, then a_i appears before a_j for any indices $i < j$ in the sorted array.

That is, the relative order of equal elements is preserved after sorting.

3.4 Minimizing Number of Swaps

The goal of minimizing the number of swaps is to reduce the overall cost of rearrangement. In this approach, elements are viewed as boxes, where some boxes are more costly to move than others.

Boxes that are difficult or expensive to move are placed into their correct positions as early as possible, so that they participate in fewer swap operations. Easier-to-move boxes are used to adjust the remaining positions, thereby minimizing the total number of swaps.

3.5 Examples of Comparison-Based Sorting Algorithms

- Insertion Sort
- Bubble Sort

- Selection Sort
- Quick Sort
- Merge Sort
- Heap Sort

3.6 Non-Comparison-Based Sorting Algorithms

- Counting Sort
- Radix Sort
- Bucket Sort

4 Algorithm Evaluation Criteria

Algorithm evaluation criteria are used to assess the quality and effectiveness of an algorithm. These criteria help in comparing different algorithms beyond time and space complexity.

- **Simplicity:** The algorithm should be easy to understand, implement, and maintain.
- **Robustness:** The algorithm should handle unexpected or extreme inputs gracefully without failure.
- **Correctness:** The algorithm must always produce the correct output for all valid inputs.

Thus, when checking the correctness and efficiency of an algorithm, it is important to clearly specify the basis on which comparisons are made.